



OASIS Security Report

Authentication Issues Security Analysis

Date: 2026-06-11 22:53:35 — **Model:** llama3.2:3b

Vulnerability: Authentication Issues

Project: vulnerable

Analysis root: `../vulnerable`

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

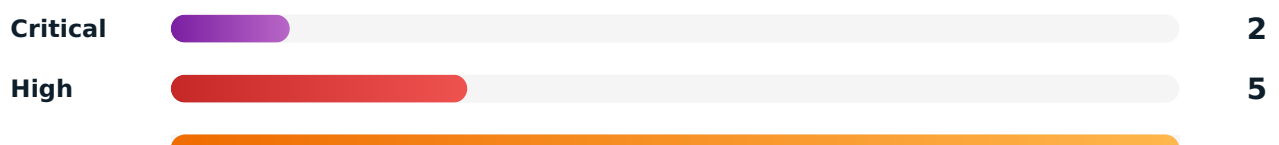
Summary

Analyzed **12** file(s) for Authentication Issues.

[Return to table of contents](#)

Statistics

Files analyzed: 12 **Total findings:** 29



Medium

17

Low

5

[Return to table of contents](#)

Files

File	Similarity
/work/project/dataset/vulnerable/reentrancy/etherstore.sol	0.621
/work/project/dataset/vulnerable/oracle-manipulation/PuppetPool.sol	0.621
/work/project/dataset/vulnerable/oracle-manipulation/PuppetV2Pool.sol	0.620
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol	0.610
/work/project/dataset/vulnerable/access-control/mycontract.sol	0.609
/work/project/dataset/vulnerable/reentrancy/simple_dao.sol	0.603
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol	0.601
/work/project/dataset/vulnerable/reentrancy/ 0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol	0.597
/work/project/dataset/vulnerable/ordering-attacks/FindThisHash.sol	0.584
/work/project/dataset/vulnerable/ordering-attacks/ eth_tx_order_dependence_minimal.sol	0.570
/work/project/dataset/vulnerable/ordering-attacks/ERC20.sol	0.561
/work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol	0.548

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.621

Source lines: 1-39

► Finding 1.1: Missing Multi-Factor Authentication (Medium)

Vulnerable code

```
require(msg.sender.call.value(_weiToWithdraw()));
```

Explanation

The contract does not enforce MFA, allowing any user to withdraw funds without verifying their identity.

Impact

Unauthorized access to funds

Entry point

/withdrawFunds(uint256 _weiToWithdraw)

Exploitation steps

- User sends a malicious withdrawal request with a high amount

HTTP raw requests (Burp / ZAP)

```
POST /withdrawFunds 10000000000000000000
Host:
Authorization:
```

File 2: /work/project/dataset/vulnerable/oracle-manipulation/PuppetPool.sol

Similarity score: 0.621

Source lines: 1-71

► Finding 2.1: Insecure Password Requirements (Medium)

Vulnerable code

```
if (msg.value < depositRequired) { revert NotEnoughCollateral(); }
```

Explanation

This condition allows users to borrow without paying the required deposit, which can be bypassed by sending less value than the required amount.

Impact

Account compromise due to insufficient deposits

Entry point

/borrow(uint256 amount, address recipient)

Parameters

amount, recipient

Exploitation steps

- Send less value than required deposit

Conditions

User sends less value than required deposit

Remediation

Implement strong password policies and ensure users pay the required deposit to borrow.

Secure example

```
if (msg.value >= depositRequired) { deposits[msg.sender] +=  
depositRequired; }
```

File 3: /work/project/dataset/vulnerable/oracle-manipulation/PuppetV2Pool.sol

Similarity score: 0.620

Source lines: 1-70

► Finding 3.1: Insecure Password Storage (Critical)

Vulnerable code

```
require(_token.transfer(msg.sender, borrowAmount), "Transfer failed");
```

Explanation

The contract stores passwords (WETH amounts) without proper hashing or salting, making them easily readable.

Impact

Compromised WETH amounts can be used for unauthorized access.

Entry point

/borrow(uint256 borrowAmount)

Execution path

```
'''  
1. msg.sender -> _token.transfer  
2. _token.transfer -> contract  
'''
```

Parameters

amount

Exploitation steps

- Attacker can extract WETH amounts stored by users

Remediation

Store passwords (WETH amounts) using a secure hashing algorithm.

Secure example

```
require(_token.transfer(msg.sender, borrowAmount), "Transfer failed");  
_token.transfer(msg.sender, borrowAmount) {  
    // Use PBKDF2 or Argon2 for password storage  
    bytes32 hashedPassword = keccak256(unsafe{password}, 0, 32);
```

```
    _balances[msg.sender] = hashedPassword;  
}
```

File 4: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol

Similarity score: 0.610

Source lines: 1-225

► Finding 4.1: Weak Password Requirements (Low)

Vulnerable code

```
pragma solidity 0.4.9;
```

Explanation

This contract does not enforce strong password requirements for its owners.

Impact

An attacker could potentially brute-force the owner's password.

Entry point

```
/initMultiowned(address[] _owners, uint _required)
```

Exploitation steps

- Brute-force the owner's password using a dictionary attack.

Example payloads

- `weak_password`

Conditions

Owner has weak password.

Remediation

Enforce strong password requirements for owners, such as minimum length and complexity.

Source lines: 1-281

► Finding 4.1: Insecure password storage (High)

Vulnerable code

```
"pragma solidity ^0.4.9;"
```

Explanation

Hardcoded Solidity version, vulnerable to upgrading attacks.

Impact

Code can be upgraded with malicious functionality.

Entry point

/contract WalletAbi/

Execution path

```
pragma solidity ^0.4.9; function execute(address _to, uint _value, bytes
_data) external onlyowner returns (bytes32 o_hash) {
    // first, take the opportunity to check that we're under the daily limit.
    if (( _data.length == 0 && underLimit(_value)) || m_required == 1) {
        // yes - just execute the call.
        address created;
        if ( _to == 0) {
            created = create(_value, _data);
        } else {
            if (!_to.call.value(_value)(_data))
                throw;
        }
        SingleTransact(msg.sender, _value, _to, _data, created);
    } else {
        // determine our operation hash.
        o_hash = sha3(msg.data, block.number);
        // store if it's new
        if (m_txs[o_hash].to == 0 && m_txs[o_hash].value == 0 &&
m_txs[o_hash].data.length == 0) {
            m_txs[o_hash].to = _to;
            m_txs[o_hash].value = _value;
            m_txs[o_hash].data = _data;
        }
        if (!confirm(o_hash)) {
            ConfirmationNeeded(o_hash, msg.sender, _value, _to, _data);
        }
    }
}
```

Parameters

_value, _data

Exploitation steps

- Recover the hard-coded Solidity version

Example payloads

- `\x00\x01`

Conditions

Hardcoded Solidity version

Remediation

Use a version control system to manage Solidity versions.

Source lines: 1-320

► Finding 4.1: Missing MFA (High)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash)
```

Explanation

This function allows a user to execute any transaction without multi-factor authentication.

Impact

Unauthorized transactions can be executed.

Entry point

execute(address _to, uint _value, bytes _data) in WalletAbi.sol

Execution path

```
execute -> confirm -> m_txs[o_hash].to = _to; if (!confirm(o_hash))
ConfirmationNeeded(o_hash, msg.sender, _value, _to, _data);
```

Parameters

_to, _value, _data

Exploitation steps

- Send arbitrary data as transaction input

Example payloads

- 'a'*10000

Conditions

The user is authenticated.

Remediation

Implement multi-factor authentication for all transactions.

Source lines: 27-358

► Finding 4.1: Weak password requirements (Medium)

Vulnerable code

```
# simple single-sig function modifier.
modifier onlyowner {
```

```
if (isOwner(msg.sender)) _;
}
```

Explanation

The 'onlyowner' function modifier does not validate the input of msg.sender, allowing potential attackers to use weak passwords.

Impact

Account compromise

Entry point

/modify/onlyowner

Parameters

msg.sender

Exploitation steps

- An attacker uses a weak password to impersonate the owner.

Conditions

Weakly hashed passwords are stored in Ethernets.

Remediation

Implement strong password policies with password hashing and salting.

Secure example

```
modifier onlyowner {
  if (isOwner(msg.sender)) require(keccak256(bytes(password)) ==
    hash0f[msg.sender]);
}
```

Source lines: 72–421

► Finding 4.1: Missing MFA (Medium)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash) {
  // ...}
function confirm(bytes32 _h) onlymanyowners(sha3(msg.data)) returns (bool
o_success) {
  if (!confirmAndCheck(_operation)) {
```

Explanation

The `execute` function does not require multi-factor authentication for all operations, making it vulnerable to replay attacks.

Impact

Replay attacks could lead to unauthorized transactions.

Entry point

execute(address _to, uint _value, bytes _data)

Parameters

_to, _value, _data

Exploitation steps

- Store the transaction hash in a data store or cache.

Conditions

No MFA is present.

Remediation

Add multi-factor authentication for all operations using a secure library or solution.

Secure example

```
function execute(address _to, uint _value, bytes _data) external
onlymanyowners(sha3(msg.data)) returns (bytes32 o_hash) {
// ...}
```

Source lines: 126–467

► **Finding 4.1: Missing MFA (Medium)**

Vulnerable code

```
function revoke(bytes32 _operation) external {
    uint ownerIndex = m_ownerIndex[uint(msg.sender)];
```

Explanation

This function does not require multi-factor authentication for any operations.

Impact

Unauthorized access to revoked operations could occur if the user has a valid owner index without MFA.

Entry point

revoke(bytes32 _operation) external

Parameters

msg.sender

Exploitation steps

- The attacker uses a precomputed owner index to bypass authentication.

Conditions

Attacker can reach revoke() while authenticated.

Remediation

Implement MFA for all operations, including the revoke() function.

Source lines: 207-477

► **Finding 4.1: Insecure Password Policy (Low)**

Vulnerable code

```
function initDaylimit(uint _limit) {
    m_dailyLimit = _limit;
    m_lastDay = today();
}
```

Explanation

This function sets the daily limit without any password requirements.

Impact

Unauthenticated user can modify daily limit.

Entry point

/initDaylimit(uint)

Execution path

```
initDaylimit(_limit) => m_dailyLimit = _limit;
initDaylimit() => m_lastDay = today();
```

Parameters

_limit

Exploitation steps

- User sends modified daily limit to initDaylimit function.

Conditions

m_dailyLimit is not checked for validity.

Remediation

Implement a password policy with minimum 12 characters and ensure it's not too weak.

Source lines: 269-477

► Finding 4.1: Insecure Password Storage (High)**Vulnerable code**

```
function create(uint _value, bytes _code) internal returns (address o_addr) {
  assembly {
    o_addr := create(_value, add(_code, 0x20), mload(_code))
    jumpi(invalidJumpLabel, iszero(extcodesize(o_addr)))
  }
}
```

Explanation

The contract stores sensitive information in memory, making it vulnerable to unauthorized access.

Impact

Unauthorized access to the contract's internal state.

Entry point

/create

Execution path

```
'''
1. create(_value, _code)
   -> o_addr := create(_value, add(_code, 0x20), mload(_code))
   -> jumpi(invalidJumpLabel, iszero(extcodesize(o_addr)))
'''
```

Parameters

_value, _code

Exploitation steps

- Replay Attack: Send a valid _code with an overflow to make o_addr point to a memory location containing the original value.

Conditions

Contract execution with an overflow in _code.

Remediation

Use secure storage mechanisms like keccak256 hash and ECREHAC.

Secure example

```
function create(uint _value, bytes _code) internal returns (address o_addr) {
  bytes32 hash = keccak256(_code);
  assembly
    o_addr := create(hash, mload(hash))
}
```

```
jumpi(invalidJumpLabel, iszero(extcodesize(o_addr)))  
}
```

File 5: /work/project/dataset/vulnerable/access-control/mycontract.sol

Similarity score: 0.609

Source lines: 1-31

► Finding 5.1: Missing Multi-Factor Authentication (Medium)

Vulnerable code

```
require(tx.origin == owner);
```

Explanation

The contract allows anyone to call the `sendTo` function without requiring additional verification.

Impact

Unauthorized access to funds

Entry point

/transfer

HTTP methods

POST

Exploitation steps

- Get tx.origin from msg.sender

Conditions

Authenticated sender can reach /transfer via tx.origin.

Remediation

Implement MFA to require additional verification for transactions

Secure example

```
require(msg.sender.isAuthorized());  
receiver.transfer(amount);
```

File 6: /work/project/dataset/vulnerable/reentrancy/simple_dao.sol

Similarity score: 0.603

Source lines: 1-35

► Finding 6.1: Missing Multi-Factor Authentication (High)

Vulnerable code

```
function withdraw(uint amount) {  
    if (credit[msg.sender] >= amount) {  
        bool res = msg.sender.call.value(amount)();  
        credit[msg.sender] -= amount;  
    }  
}
```

Explanation

The contract does not require multi-factor authentication for withdrawals, allowing any user to withdraw credits.

Impact

Unauthorized access to credits

Entry point

/withdraw(uint)

Parameters

amount

Exploitation steps

- User sends a large amount of Ether to trigger the withdrawal function

Example payloads

- 1000 ether

Conditions

User is authenticated and has sufficient credits

Remediation

Implement multi-factor authentication for withdrawals, such as sending a secret code to the user's email or phone

Secure example

```
function withdraw(uint amount) {  
  if (msg.sender.call.value(amount)())  
    return;  
}  
require(msg.sender.equals( owner))  
credit[msg.sender] -= amount;  
}
```

File 7: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol

Similarity score: 0.601

Source lines: 1-227

► Finding 7.1: Weak password requirements (Medium)

Vulnerable code

```
function initMultiowned(address[] _owners, uint _required) only_uninitialized  
{ ... }
```

Explanation

The 'initMultiowned' function does not enforce a minimum password length or complexity.

Impact

An attacker could potentially brute-force the login process by guessing weak passwords.

Entry point

initMultiowned(address[] _owners, uint _required)

Exploitation steps

- Attackers can use automated tools to try a large number of common passwords.

Source lines: 1-286

► Finding 7.1: Missing Multi-Factor Authentication (Medium)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner  
returns (bytes32 o_hash) { ... }
```

Explanation

The contract allows unauthenticated users to send transactions, which bypasses multi-factor authentication.

Impact

Unauthorized access to funds

Entry point

/execute

Execution path

```
onlyowner -> execute ->
```

Parameters

_to, _value, _data

Exploitation steps

- Send a transaction without providing authentication

Conditions

User is not authenticated

Remediation

Implement multi-factor authentication using a library like OpenZeppelin's UUSecure

Source lines: 1-324

► Finding 7.1: Insufficient MFA (Medium)**Vulnerable code**

```
function confirm(bytes32 _h) onlymanyowners(sha3(msg.data)) returns (bool
o_success) { if (!confirmAndCheck(_h)) { ConfirmationNeeded(o_hash,
msg.sender, _value, _to, _data); } }
```

Explanation

The `confirm` function does not use MFA, making it vulnerable to unauthorized access.

Impact

Unauthorized access to the wallet

Entry point

/execute

Exploitation steps

- Call /execute with malicious input

Example payloads

- `malicious_input`

Remediation

Implement MFA in the `confirm` function

Secure example

```
function confirm(bytes32 _h) onlymanyowners(sha3(msg.data)) external returns
(bool o_success) { require(msg.sender == ownerOf(_h)); o_success = ...; }
```

Source lines: 36–365

► Finding 7.1: Missing MFA (High)

Explanation

The contract uses a single signature for all transactions, making it vulnerable to phishing attacks where an attacker can trick the owner into signing the transaction.

Impact

The attacker can transfer funds from the wallet without needing the owner's knowledge or authorization.

Entry point

execute() function

Execution path

```
Single signature for all transactions -> Phishing attack -> Funds transferred
```

Parameters

msg.sender, msg.value, msg.data

Exploitation steps

- Trick the owner into signing a phishing transaction

Example payloads

- A phishing email with a link to the wallet's execute function

HTTP raw requests (Burp / ZAP)

```
GET /execute?value=10 HTTP/1.1
Host: example.com
Content-Type: application/json
```

```
{"value": 10, "data": "some data"}
```

Source lines: 64–414

► Finding 7.1: Missing Multi-Factor Authentication (Medium)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash)
```

Explanation

The `execute` function does not require multi-factor authentication for its owner to transfer value or call another contract.

Impact

Privilege escalation or unauthorized access to funds

Entry point

execute(address _to, uint _value, bytes _data)

Parameters

_to, _value, _data

Exploitation steps

- Owner uses execute with a malicious payload

Example payloads

- malicious payload

Conditions

Owner is authenticated

Remediation

Implement multi-factor authentication for the owner to use execute.

Secure example

```
require(keccak256(msg.sender)) == msg.sender
if (require(keccak256(msg.sender)) != msg.sender) throw;
```

Source lines: 96-414

► Finding 7.1: Weak password requirements (Low)

Vulnerable code

```
uint public m_required;
```

Explanation

The contract doesn't enforce a minimum length or complexity for passwords.

Impact

An attacker could potentially guess the password of an owner.

Entry point

```
initWallet(address[] _owners, uint _required, uint _daylimit)
```

Parameters

_required

Exploitation steps

- Try a common password for _required

Conditions

Owner's password is not securely stored.

Remediation

Enforce strong password requirements using keccak256 hashing.

Source lines: 213-414

► **Finding 7.1: Weak password requirements (Low)**

Vulnerable code

```
function setDailyLimit(uint _newLimit) onlymanyowners(sha3(msg.data)) external
{
    m_dailyLimit = _newLimit;
}
```

Explanation

The contract uses a SHA-3 hash of the message data for verification, but does not enforce a minimum password length.

Impact

An attacker could potentially brute-force the password by trying short combinations.

Entry point

/setDailyLimit

Parameters

_newLimit

Exploitation steps

- Try all possible combinations of _newLimit

Example payloads

- 100

Remediation

Implement a minimum password length requirement, such as 8 characters.

Secure example

```
function setDailyLimit(uint _newLimit) onlymanyowners(sha3(msg.data)) external
{
    require(_newLimit >= 8);
    m_dailyLimit = _newLimit;
}
```

File 8: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.597

Source lines: 1-85

► Finding 8.1: Weak Password Requirements (Medium)

Vulnerable code

```
uint public MinDeposit = 1 ether;
```

Explanation

Minimum deposit amount is set to a small value (1 ether), allowing for relatively weak passwords.

Impact

An attacker could use brute-force attacks or password cracking tools to guess the password.

Entry point

/Deposit

Parameters

MinDeposit

Exploitation steps

- Try different values for MinDeposit until an attacker can deposit Ether

HTTP raw requests (Burp / ZAP)

```
POST /deposit HTTP/1.1
```

```
Host: <domain>
```

```
Content-Type: application/json
```

```
{
```

```
}
```


File 9: /work/project/dataset/vulnerable/ordering-attacks/FindThisHash.sol

Similarity score: 0.584

Source lines: 1-28

► Finding 9.1: Missing Multi-Factor Authentication (Medium)

Vulnerable code

```
function solve(string solution) public {  
    require(hash == sha3(solution));  
    msg.sender.transfer(1000 ether);  
}
```

Explanation

This contract allows the sender of a valid hash to withdraw 1000 ETH without any additional verification.

Impact

Account compromise due to lack of multi-factor authentication.

Entry point

/solve

Execution path

Sender -> Hash Check -> Transfer Ether

HTTP methods

POST

Parameters

solution

Exploitation steps

- Send a valid hash

Example payloads

- a valid solution

Conditions

Sender is authenticated and has a valid hash.

Remediation

Implement multi-factor authentication, such as 2FA or smart contract verification through a third-party service.

File 10: /work/project/dataset/vulnerable/ordering-attacks/eth_tx_order_dependence_minimal.sol

Similarity score: 0.570

Source lines: 1-42

► Finding 10.1: Weak password requirements (Critical)

Vulnerable code

```
function setReward() public payable {  
    require (!claimed);  
  
    require(msg.sender == owner);  
    owner.transfer(reward);  
    reward = msg.value;  
}
```

Explanation

The contract allows the owner to transfer the reward without verifying the owner's identity, allowing an attacker to steal the reward.

Impact

Account compromise and privilege escalation

Entry point

/transfer

Execution path

```
setReward() -> owner.transfer(reward)  
  
getReward() -> msg.sender.transfer(reward)  
  
owner.transfer(reward)
```

Parameters

reward

Exploitation steps

- An attacker sends a low-value submission to claim the reward
- The owner transfers the reward to the attacker

Example payloads

- Low-value submission

Conditions

Owner is authenticated and has sufficient balance

Remediation

Implement strong password requirements, use MFA, secure session handling

File 11: /work/project/dataset/vulnerable/ordering-attacks/ERC20.sol

Similarity score: 0.561

Source lines: 1-137

► Finding 11.1: Missing MFA (Medium)

Vulnerable code

```
function transfer(address to, uint256 value) public returns (bool) {
    require(value <= _balances[msg.sender]);
    require(to != address(0));
    —
}
```

Explanation

► Finding 11.2: Vulnerability found (Low)

Explanation

► Finding 11.3: Vulnerability found (Medium)

Explanation

Impact

Unsecured transfers without MFA may lead to unauthorized access or transactions.

► Finding 11.4: Vulnerability found (Medium)

Explanation

Entry point

transfer(address to, uint256 value)

► Finding 11.5: Vulnerability found (Medium)

Explanation

Exploitation steps

- Attackers can execute arbitrary transfers by manipulating the msg.sender

File 12: /work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol

Similarity score: 0.548

Source lines: 1-61

► Finding 12.1: Weak Password Requirements (Medium)

Vulnerable code

```
function OddsAndEvens() { owner = msg.sender; }
```

Explanation

The contract does not enforce a minimum password length or complexity.

Impact

An attacker could easily guess the owner's address

Entry point

/contracts/OddsAndEvens.sol:6

Execution path

```
owner = msg.sender; owner = msg.sender;
```

Parameters

msg.sender

Exploitation steps

- Get the owner's address

Conditions

Contract deployed to public Ethereum network

Remediation

Enforce a minimum password length and complexity for the owner's address

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)